



T E R R A F O R M

Complete Notes & Interview Guide

State Management · Modules · Workspaces · EKS

Commands · File Reference · Scenario-Based Interview Q&A

Terraform — Complete Notes & Interview Guide

State management · Modules · Workspaces · EKS · Commands · Scenario-based Q&A

1. What Terraform Is

Terraform is HashiCorp's Infrastructure-as-Code (IaC) tool. You describe the infrastructure you want in declarative configuration files (`.tf`), and Terraform works out the sequence of API calls needed to make reality match that description.

Declarative, not imperative. You write *what* you want, not the steps to get there. A shell script says "create a VPC, then a subnet, then an instance." Terraform says "this is the world I want" and figures out the order, including what needs to be destroyed and recreated versus updated in place.

Why teams adopt it

Problem with console clicking	Terraform's fix
Not repeatable	Same <code>.tf</code> files produce the same infrastructure every time
No audit trail	Configuration lives in Git; every change is a diff and a commit
No preview	<code>terraform plan</code> shows exactly what will change before it happens
Hard to reuse	Modules package a resource pattern once, call it many times
Vendor lock-in to a console	One workflow across AWS, Azure, GCP, Kubernetes, and 3000+ other providers

Terraform vs. Ansible/Chef/Puppet — Terraform is a *provisioning* tool (creates infrastructure); Ansible etc. are *configuration management* tools (configure software on infrastructure that already exists). They're commonly used together: Terraform builds the VM, Ansible installs the app on it.

Terraform vs. CloudFormation/ARM/Deployment Manager — those are cloud-specific. Terraform is cloud-agnostic and manages state itself rather than relying on the cloud provider's own state tracking, which is why it can orchestrate resources across multiple providers in one apply.

2. Core Workflow

write `.tf` files → `init` → `plan` → `apply` → (later) `destroy`

Command	What it does	Talks to cloud APIs?
<code>terraform init</code>	Downloads providers/modules, sets up the backend	No (just downloads)
<code>terraform fmt</code>	Rewrites files to canonical style	No
<code>terraform validate</code>	Checks syntax and internal consistency	No
<code>terraform plan</code>	Computes and shows the diff between state and config	Yes, read-only
<code>terraform apply</code>	Executes the plan	Yes, read-write
<code>terraform destroy</code>	Removes everything Terraform tracks in this state	Yes, read-write

The plan is not a guarantee. Between `plan` and `apply`, someone else's change or a slow-changing attribute (like an AMI resolved via a `data` source) can make the actual apply differ slightly. Saving the plan to a file and applying *that exact file* removes this gap — see `-out` below.

3. Complete Command Reference

3.1 Setup and lifecycle

```

terraform init                # initialize working directory
terraform init -upgrade       # upgrade providers/modules to latest allowed version
terraform init -reconfigure   # reinitialize backend, ignore saved config
terraform init -migrate-state # move state when backend config changes
terraform init -backend-config=x.hcl # supply backend settings at init time
terraform init -backend=false  # skip backend, useful for `validate` in CI

terraform fmt                 # format files in current directory
terraform fmt -recursive     # format every .tf file in the tree
terraform fmt -check          # exit non-zero if anything is unformatted (CI)
terraform fmt -diff           # show what would change

terraform validate            # syntax + internal consistency check
terraform validate -json     # machine-readable output

terraform version             # Terraform + provider versions
terraform providers           # show required providers for this config
terraform providers lock      # write exact provider hashes to the lock file
terraform providers schema -json # full schema of every provider resource/data source

```

3.2 Planning and applying

```

terraform plan                # show proposed changes
terraform plan -out=tfplan    # save the plan to a file
terraform plan -destroy       # preview what destroy would do
terraform plan -var="env=prod" # override one variable
terraform plan -var-file="prod.tfvars" # load variables from a file
terraform plan -target=aws_instance.web # narrow scope to one resource (debugging only)
terraform plan -refresh=false  # skip refreshing state against real infra (faster, riskier)
terraform plan -replace=aws_instance.web # force a resource to be destroyed + recreated

terraform apply               # interactive: shows plan, asks to confirm
terraform apply -auto-approve # skip confirmation (CI pipelines)
terraform apply tfplan        # apply an exact saved plan, no new diff computed
terraform apply -var="env=prod"
terraform apply -parallelism=5 # limit concurrent resource operations (default 10)

terraform destroy             # tear down everything in this state
terraform destroy -auto-approve
terraform destroy -target=aws_instance.web # destroy one resource only

```

3.3 State inspection and surgery

```

terraform state list          # every resource address Terraform tracks
terraform state list 'aws_instance.*' # filter by pattern
terraform state show aws_instance.web # full attributes of one resource, as Terraform sees them
terraform state pull          # print raw state JSON (or redirect to a backup file)
terraform state push newstate.tfstate # overwrite remote state (dangerous, rarely needed)

terraform state mv aws_instance.a aws_instance.b # rename in state, no resource recreation
terraform state mv module.old module.new         # move into/out of a module
terraform state rm aws_instance.web               # forget a resource (does NOT delete the real thing)

terraform import aws_s3_bucket.b my-existing-bucket # adopt existing infra into state
terraform refresh                                     # (deprecated – use `plan -refresh-only`)
terraform apply -refresh-only                         # sync state with real-world drift, no changes applied

terraform force-unlock LOCK_ID # break a stuck state lock (confirm nobody is applying first)

terraform show # human-readable dump of current state
terraform show -json # machine-readable, pipe to jq
terraform graph # dependency graph in DOT format (pipe to graphviz)

```

3.4 Workspaces

```

terraform workspace list # all workspaces, * marks current
terraform workspace show # name of current workspace
terraform workspace new dev # create and switch to a new workspace
terraform workspace select prod # switch to an existing workspace
terraform workspace delete qa # delete (must be empty / not current)

```

3.5 Modules & providers

```

terraform get # download modules referenced by config (init does this too)
terraform get -update # re-download even if already present

terraform providers mirror ./local-mirror # download providers for offline/air-gapped use

```

3.6 Output & variables

```
terraform output           # show all outputs
terraform output instance_ip # show one output
terraform output -json      # machine-readable
terraform output -raw bucket_name # unquoted, for shell scripting
```

3.7 Testing (Terraform ≥ 1.6)

```
terraform test           # run *.tftest.hcl files in ./tests or alongside config
terraform test -filter=basic.tftest.hcl # run one test file
terraform test -verbose  # show full plan/apply output per run block
```

3.8 Console & debugging

```
terraform console           # interactive REPL for expressions
                           # try: var.foo, local.bar, [for k,v in var.map : k]

TF_LOG=DEBUG terraform apply # verbose logging (TRACE, DEBUG, INFO, WARN, ERROR)
TF_LOG_PATH=./tf.log terraform apply # write log to a file
TF_VAR_region=ap-south-1 terraform plan # set a variable via environment
```

3.9 Cloud Development Kit / newer tooling (context, not required)

```
tofu ... # OpenTofu – the open-source Terraform fork, same CLI surface
```

4. Anatomy of a `.tf` File

4.1 `terraform` block

```
terraform {
  required_version = ">= 1.6.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }

  backend "s3" {
    bucket     = "my-tf-state"
    key        = "prod/terraform.tfstate"
    region     = "ap-south-1"
    dynamodb_table = "tf-lock"
    encrypt    = true
  }
}
```

- `required_version` pins the Terraform CLI version.
- `required_providers` pins provider source and version — **always set this**; an unpinned provider can upgrade under you between two `init` runs and silently change behaviour.
- `backend` cannot use variables or interpolation — values must be literal, or supplied via `-backend-config` at init time.

4.2 `provider` block

```

provider "aws" {
  region = "ap-south-1"
  profile = "my-profile"      # optional; else picked up from env / default credential chain

  default_tags {
    tags = { ManagedBy = "terraform" }
  }
}

# Multiple regions: use alias
provider "aws" {
  alias   = "us_east"
  region = "us-east-1"
}

resource "aws_acm_certificate" "cert" {
  provider = aws.us_east      # CloudFront/ACM certs must be in us-east-1
  # ...
}

```

4.3 resource block

```

resource "<PROVIDER_TYPE>" "<LOCAL_NAME>" {
  argument = value
  # ...

  lifecycle {
    create_before_destroy = true
    prevent_destroy       = false
    ignore_changes         = [tags]
    precondition {
      condition = var.instance_type != ""
      error_message = "instance_type must be set."
    }
    postcondition {
      condition = self.public_ip != ""
      error_message = "Instance did not get a public IP."
    }
  }

  depends_on = [aws_iam_role_policy_attachment.example] # explicit dependency, use sparingly
}

```

Reference it elsewhere as `<TYPE>.<LOCAL_NAME>.<ATTRIBUTE>`, e.g. `aws_instance.web.public_ip`.

4.4 variable block

```

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t3.micro"
  sensitive   = false

  validation {
    condition = contains(["t3.micro", "t3.small", "t3.medium"], var.instance_type)
    error_message = "instance_type must be a t3 family size."
  }
}

# Type constraints
variable "a" { type = string }
variable "b" { type = number }
variable "c" { type = bool }
variable "d" { type = list(string) }
variable "e" { type = map(number) }
variable "f" { type = set(string) }
variable "g" {
  type = object({
    name = string
    port = number
    tags = optional(map(string), {}) # optional() gives a default inside an object type
  })
}
variable "h" { type = any }

```

Precedence (highest wins):

1. `-var / -var-file` on the CLI
2. `*.auto.tfvars` (alphabetical order)
3. `terraform.tfvars`

4. `TF_VAR_name` environment variables
5. `default` in the `variable` block

4.5 `output` block

```
output "instance_ip" {
  description = "Public IP of the web server"
  value       = aws_instance.web.public_ip
  sensitive   = false
}
```

4.6 `locals` block

```
locals {
  name_prefix = "${var.project}-${var.env}"
  common_tags = merge({ Project = var.project }, var.extra_tags)
}
```

A `local` is a computed value the *caller can't override* — use it for anything derived from variables. A `variable` is an input; a `local` is a derived constant within the same configuration.

4.7 `data` source

```
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
}

resource "aws_instance" "web" {
  ami = data.aws_ami.ubuntu.id
  # ...
}
```

`data` reads existing information (from the provider or from another Terraform state); it does not create anything.

4.8 `module` block

```
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"
  version = "~> 5.13"           # always pin a registry module's version

  name = "my-vpc"
  cidr = "10.0.0.0/16"
}

# reference module outputs
resource "aws_instance" "web" {
  subnet_id = module.vpc.public_subnets[0]
}
```

4.9 `import` block (Terraform $\geq$ 1.5)

```
import {
  to = aws_s3_bucket.legacy
  id = "my-manually-created-bucket"
}

resource "aws_s3_bucket" "legacy" {
  bucket = "my-manually-created-bucket"
}
```

Run `terraform plan -generate-config-out=generated.tf` to have Terraform write the resource block for you — review before committing.

4.10 `moved` and `removed` blocks

```

moved {
  from = aws_instance.old_name
  to   = aws_instance.new_name
}

removed {
  from = aws_instance.retired
}

lifecycle {
  destroy = false
}

```

4.11 `check` block (Terraform ≥ 1.5)

```

check "health_check" {
  data "http" "web" {
    url = "https://${aws_instance.web.public_dns}"
  }

  assert {
    condition     = data.http.web.status_code == 200
    error_message = "Website did not return 200."
  }
}

```

A `check` reports a warning without blocking the plan — unlike a `precondition` / `postcondition`, which fails it.

5. Meta-arguments: `count`, `for_each`, `depends_on`, `lifecycle`, `provider`

5.1 `count` VS `for_each`

```

# count - indexed by POSITION
resource "aws_instance" "server" {
  count      = 3
  ami       = var.ami
  instance_type = "t3.micro"
  tags      = { Name = "server-${count.index}" }
}
# addresses: aws_instance.server[0], [1], [2]

# for_each over a set - indexed by KEY (string)
resource "aws_instance" "server" {
  for_each   = toset(["web", "api", "worker"])
  ami       = var.ami
  instance_type = "t3.micro"
  tags      = { Name = each.key }
}
# addresses: aws_instance.server["web"], ["api"], ["worker"]

# for_each over a map - key AND value
resource "aws_instance" "server" {
  for_each   = { web = "t3.micro", api = "t3.small" }
  instance_type = each.value
  tags      = { Name = each.key }
}

```

Why `for_each` is usually safer: with `count`, removing the first element of a list shifts every subsequent index down by one — Terraform sees "index 0 changed" and destroys + recreates every instance after the removed one, not just the one you meant to remove. `for_each` addresses by stable key, so removing "web" touches only "web".

Use `count` for: N identical, interchangeable copies of something, or a 0/1 toggle (`count = var.enabled ? 1 : 0`). **Use `for_each`** for: a set of named, individually-referenceable things that can grow or shrink.

5.2 `depends_on`

Terraform infers dependency order automatically from references (`aws_instance.web` referencing `aws_security_group.sg.id` creates an implicit dependency). `depends_on` is for the rare case where one resource depends on another with **no attribute reference** — e.g., an IAM policy needs to exist before an EC2 instance assumes the role, but the instance resource never reads any of the policy's attributes.

```
resource "aws_instance" "web" {
  # ...
  depends_on = [aws_iam_role_policy_attachment.example]
}
```

Overusing `depends_on` is a smell — it usually means the configuration should be restructured so the dependency is expressed through a reference instead.

5.3 lifecycle

Argument	Effect
<code>create_before_destroy = true</code>	Build the replacement before destroying the original — essential for anything serving live traffic
<code>prevent_destroy = true</code>	<code>terraform destroy</code> (or any plan that would destroy this resource) fails outright — use on state buckets, production databases
<code>ignore_changes = [tags, ami]</code>	Stop Terraform from "fixing" a value something else legitimately owns (an autoscaler, a deploy pipeline)
<code>replace_triggered_by = [aws_instance.web.id]</code>	Force replacement of this resource whenever the referenced resource changes
<code>precondition { }</code>	Fails the plan if a condition about inputs isn't met
<code>postcondition { }</code>	Fails the apply if a condition about the resulting resource isn't met

5.4 Provider meta-argument

```
resource "aws_instance" "web" {
  provider = aws.us_east # use an aliased provider instead of the default
}
```

6. Expressions & Functions Cheat-Sheet

```
# Conditional (ternary)
instance_type = var.env == "prod" ? "t3.large" : "t3.micro"

# for expression - list
names = [for u in var.users : upper(u.name)]

# for expression - map, with filter
adults = { for k, v in var.people : k => v if v.age >= 18 }

# splat expression
ids = aws_instance.web[*].id

# dynamic block - generate nested blocks from a collection
resource "aws_security_group" "sg" {
  dynamic "ingress" {
    for_each = var.ports
    content {
      from_port = ingress.value
      to_port   = ingress.value
      protocol  = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
    }
  }
}
```

Common built-in functions

Function	Example	Result
<code>merge</code>	<code>merge({a=1}, {b=2})</code>	<code>{a=1, b=2}</code>
<code>lookup</code>	<code>lookup(map, "key", "default")</code>	value or default
<code>concat</code>	<code>concat([1,2],[3])</code>	<code>[1,2,3]</code>
<code>element</code>	<code>element(list, index)</code>	value at index (wraps)
<code>join / split</code>	<code>join(",", list)</code>	string
<code>length</code>	<code>length(list)</code>	count
<code>contains</code>	<code>contains(list, "x")</code>	bool
<code>try</code>	<code>try(a.b.c, "fallback")</code>	value or fallback, no error
<code>coalesce</code>	<code>coalesce(null, "", "x")</code>	first non-null/non-empty
<code>jsonencode / jsondecode</code>	—	convert to/from JSON string
<code>file</code>	<code>file("key.pub")</code>	file contents as string
<code>templatefile</code>	<code>templatefile("tpl.tftpl", {name="x"})</code>	rendered template
<code>timestamp</code>	<code>timestamp()</code>	current UTC time (careful — changes every apply)
<code>uuid</code>	<code>uuid()</code>	random UUID (same caution)
<code>cidrsubnet</code>	<code>cidrsubnet("10.0.0.0/16", 8, 1)</code>	<code>10.0.1.0/24</code>

7. State Management — Deep Notes

What state is: a JSON file (`terraform.tfstate`) mapping resource addresses in your configuration (`aws_instance.web`) to real-world object IDs (`i-0abc123`) and their full attribute values as of the last apply/refresh.

Why it must exist at all: Terraform is declarative — your `.tf` files describe the desired end state, not the current one. Without a record of what it already built, `plan` couldn't tell "create this" from "this already exists, leave it alone."

7.1 The three problems remote state solves

1. **Single point of access.** Local state means only the person with that file on their laptop can run `apply`. An S3 (or GCS, Azure Blob, Terraform Cloud) backend puts it somewhere the whole team can reach.
2. **Concurrent writes.** Two people applying at the same moment both read the same state, both write back — the second write silently overwrites the first person's changes in the state file (the real resources still exist, but Terraform's *record* of them is gone). A locking mechanism (DynamoDB table, or S3's native `use_lockfile` in Terraform ≥ 1.10) makes the second person get a clean "error acquiring the state lock" instead.
3. **Secrets in plaintext.** State stores every attribute value verbatim, including ones marked `sensitive` in the config (sensitivity only affects CLI/log output, not what's written to the state file). An RDS master password, a generated TLS key — all sit in state in the clear. Mitigate with encryption at rest, strict access controls, and never committing state to Git.

7.2 Backend configuration

```
terraform {
  backend "s3" {
    bucket = "my-tf-state"
    key    = "network/terraform.tfstate"
    region = "ap-south-1"
    dynamodb_table = "tf-lock"      # classic locking
    # use_lockfile = true           # Terraform >= 1.10 alternative: lock via S3 object, no DynamoDB needed
    encrypt      = true
  }
}
```

Backend blocks cannot reference variables. To parameterize per-environment, use partial configuration:

```
terraform init -backend-config="bucket=my-tf-state" -backend-config="key=prod/terraform.tfstate"
# or
terraform init -backend-config=backends/prod.hcl
```

7.3 Reading another stack's state

```
data "terraform_remote_state" "network" {
  backend = "s3"
  config = {
    bucket = "my-tf-state"
    key    = "network/terraform.tfstate"
    region = "ap-south-1"
  }
}

resource "aws_instance" "app" {
  subnet_id = data.terraform_remote_state.network.outputs.private_subnet_ids[0]
}
```

This is how large infrastructures split into independently-applied stacks without losing the wiring between them.

7.4 State surgery — when to use what

Situation	Command
Rename a resource without recreating it	<code>terraform state mv</code> or a <code>moved</code> block (preferred — reviewable in a PR)
Terraform should stop managing something, but leave the real resource alone	<code>terraform state rm</code> or a <code>removed</code> block
Adopt infrastructure that already exists	<code>terraform import</code> or an <code>import</code> block
State thinks something exists but it doesn't (or vice versa)	<code>terraform apply -refresh-only</code> , review, then apply normally
A lock is stuck because a process died mid-apply	<code>terraform force-unlock <LOCK_ID></code> — only after confirming nobody is actually applying

Golden rules: never hand-edit `terraform.tfstate`. Never commit it to version control. One backend `key` per stack — sharing a key between two stacks means the second `init` silently overwrites the first stack's state.

8. Modules — Deep Notes

A module is simply a directory of `.tf` files. Every configuration you write is a module — the one at the top is the **root module**; anything it calls via a `module` block is a **child module**.

8.1 Standard structure

```
modules/s3/
├── main.tf          # resources
├── variables.tf     # inputs — typed, described, validated
├── outputs.tf       # exports — what a caller can use
├── versions.tf      # required_version + provider constraints
└── README.md       # usage example + inputs/outputs tables
```

8.2 Rules that come up constantly in real codebases and interviews

1. **No `provider` block inside a module.** Modules inherit whichever provider(s) the *caller* configured. A hardcoded provider block inside a module makes it impossible to reuse in a multi-region or multi-account setup.
2. **Pin tightly in the root, loosely in the module.** Root: `version = "~> 6.0"`. Shared module: `version = ">= 5.0"`. The root is what actually gets applied and should be reproducible; the module should stay compatible with a wider range so it doesn't force unnecessary upgrades on every caller.
3. **Output generously.** Adding an output later is free. A caller who needs a value you didn't export has to fork the module or reach into implementation details they shouldn't touch.
4. **Validate inputs at the boundary** with `validation` blocks — failing during `plan` in milliseconds beats failing two minutes into `apply` with half-built infrastructure.
5. **Keep nesting shallow.** Root → module is fine. Root → module → module → module means nobody can trace where a value actually came from.
6. **Version remote modules explicitly.** `git::https://github.com/org/repo.git/s3?ref=v1.2.0` — an unpinned `ref` means the module can change under you between two `init` runs.

8.3 Module sources

```
source = "../modules/s3"           # local path, relative to caller
source = "terraform-aws-modules/vpc/aws" # Terraform Registry
source = "git::https://github.com/org/repo.git//s3?ref=v1.2.0" # git, subdirectory, pinned tag
source = "git::git@github.com:org/repo.git?ref=main"           # git over SSH
```

8.4 When *not* to write a module

A module that wraps a single resource and passes every argument straight through adds indirection with no benefit — write the resource directly. Reach for a module when there's a genuine *pattern* worth naming: "an S3 bucket that's always encrypted, versioned, and blocks public access" is a pattern; "a bucket" is not.

9. Workspaces — Deep Notes

A workspace is a named, separate **state file** for the same configuration — same `.tf` files, same providers, same resource definitions, different state.

```
terraform workspace new dev
terraform workspace new stg
terraform workspace new prod
terraform workspace select prod
terraform workspace show
```

9.1 Where state lives per workspace

Backend	Path for the <code>dev</code> workspace
local	<code>terraform.tfstate.d/dev/terraform.tfstate</code>
S3	<code>s3://<bucket>/env:/dev/<key></code>

`default` is special — it has **no** `env:/` prefix. Leaving it unused and naming real environments explicitly (`dev`, `stg`, `prod`) avoids accidentally treating `default` as a real environment.

9.2 Driving per-environment differences

```
locals {
  env_config = {
    dev = { instance_type = "t3.micro", instance_count = 1 }
    stg = { instance_type = "t3.micro", instance_count = 1 }
    prod = { instance_type = "t3.small", instance_count = 2 }
  }
  cfg = lookup(local.env_config, terraform.workspace, local.env_config["dev"])
}

resource "aws_instance" "app" {
  count           = local.cfg.instance_count
  instance_type  = local.cfg.instance_type
  tags           = { Name = "app-${terraform.workspace}-${count.index}" }
}
```

9.3 Workspaces vs. directory-per-environment

	Workspaces	Separate root directories
Code duplication	None	One copy per environment
Adding an environment	<code>terraform workspace new qa</code>	Copy a folder
Blast radius of a config typo	Every environment	Just the one directory
Different AWS accounts per env	Not supported cleanly	Natural
Different backends per env	Not supported	Natural
Structurally different environments	Awkward	Natural

Rule of thumb: workspaces when environments differ only in *size/scale* and share an account and backend. Separate directories (often with Terragrunt layered on top) when environments differ in *structure*, span accounts, or need

different backends — which describes most real production setups, which is why many teams outgrow workspaces.

9.4 Common workspace mistakes

1. Forgetting to `select` before applying — you think you're in `dev`, you're actually still in `prod`.
2. Building globally-unique resource names without including `terraform.workspace` — two workspaces then collide on the same S3 bucket name and the second apply fails.
3. Treating `default` as production.
4. A typo'd workspace name (`production`) silently creates a *new*, empty environment rather than erroring — guard against this with a `precondition` that checks the workspace name against a known list.

10. EKS on Terraform — Deep Notes

10.1 Minimal but production-shaped module call

```
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"
  version = "~> 5.13"

  name = "my-cluster-vpc"
  cidr = "10.0.0.0/16"

  azs          = ["ap-south-1a", "ap-south-1b", "ap-south-1c"]
  private_subnets = ["10.0.1.0/24", "10.0.2.0/24", "10.0.3.0/24"]
  public_subnets = ["10.0.101.0/24", "10.0.102.0/24", "10.0.103.0/24"]

  enable_nat_gateway = true
  single_nat_gateway = true # cost saving; one per AZ for HA

  public_subnet_tags = { "kubernetes.io/role/elb" = "1" }
  private_subnet_tags = { "kubernetes.io/role/internal-elb" = "1" }
}

module "eks" {
  source = "terraform-aws-modules/eks/aws"
  version = "~> 21.0"

  name = "my-cluster"
  kubernetes_version = "1.33"

  endpoint_public_access = true
  enable_cluster_creator_admin_permissions = true

  vpc_id = module.vpc.vpc_id
  subnet_ids = module.vpc.private_subnets

  addons = {
    vpc-cni = { most_recent = true }
    kube-proxy = { most_recent = true }
    coredns = { most_recent = true }
  }

  eks_managed_node_groups = {
    general = {
      instance_types = ["t3.medium"]
      min_size = 2
      max_size = 5
      desired_size = 2
      capacity_type = "SPOT"
    }
  }
}
```

10.2 Concepts worth knowing cold

- **Three subnet tiers.** Public (NAT gateways, internet-facing load balancers), private (worker nodes — outbound only via NAT), intra (no internet route at all — a good home for control-plane ENIs).
- **Subnet tags are load-bearing, not decorative.** `kubernetes.io/role/elb` and `kubernetes.io/role/internal-elb` tell the AWS Load Balancer Controller which subnets to place a `Service` of type `LoadBalancer` into. Miss them and the service hangs in `<pending>` forever.
- `enable_cluster_creator_admin_permissions` adds whoever ran `apply` as a cluster admin via an EKS access entry. Without it, you get a cluster your own credentials can't `kubectl` into — EKS access entries replaced the older `aws-auth` ConfigMap approach.

- **SPOT vs ON_DEMAND capacity.** SPOT is 60–70% cheaper but interruptible with roughly two minutes' notice — fine for stateless workloads, wrong for anything holding local, unreplicated state.
- **Version pinning matters more here than almost anywhere else.** The `terraform-aws-modules/eks/aws` module made breaking argument renames between major versions (`cluster_addons` → `addons`, `cluster_name` → `name` in v21). An unpinned `source` means the next `init` silently upgrades and the next `plan` fails or, worse, proposes destructive changes.

10.3 Connecting `kubernetes` / `helm` providers to the cluster you just created

```
provider "kubernetes" {
  host      = module.eks.cluster_endpoint
  cluster_ca_certificate = base64decode(module.eks.cluster_certificate_authority_data)

  exec {
    api_version = "client.authentication.k8s.io/v1beta1"
    command     = "aws"
    args        = ["eks", "get-token", "--cluster-name", module.eks.cluster_name]
  }
}
```

10.4 Destroy gotcha

Kubernetes creates AWS resources Terraform doesn't know about — a `Service` of type `LoadBalancer` provisions a real ELB. `terraform destroy` will hang waiting for the subnet to delete while that ELB still references it.

```
kubectl delete svc --all --all-namespaces
kubectl delete ingress --all --all-namespaces
terraform destroy
```

11. Scenario-Based Interview Questions

Real interviews for DevOps/Cloud/SRE roles rarely ask "what is a variable." They describe a situation and want to hear how you reason through it. Each question below includes a strong sample answer — read the reasoning, not just the conclusion.

State Management

Q1. Your teammate ran `terraform apply` and it hung for ten minutes before failing with "Error acquiring the state lock." What's happening, and what do you check before force-unlocking?

Someone (or some CI job) already holds the lock — either genuinely mid-apply, or a process that crashed or was killed without releasing it cleanly. Before touching `force-unlock`:

1. Check with the team / check CI for a currently-running pipeline against this state.
2. Look at the lock info in the error message — it includes who acquired it and when (`Info`, `Created` timestamp). A lock from 3 minutes ago is probably a real, in-progress apply. A lock from 6 hours ago, especially outside working hours, is probably orphaned.
3. Only then run `terraform force-unlock <LOCK_ID>`.

Force-unlocking while someone is genuinely applying can let two applies race and corrupt state — this is exactly the failure mode locking exists to prevent, so treat the override with real caution.

Q2. A junior engineer accidentally ran `terraform destroy` against the wrong workspace and took down a chunk of production. How would you have prevented this, and what's your first move afterward?

Prevention, layered:

- `prevent_destroy = true` in the `lifecycle` block on anything unrecoverable (state buckets, prod databases, prod's own state-locking table).
- Separate AWS accounts per environment (not just workspaces) so a `destroy` in the wrong *context* physically cannot reach production resources, since the credentials wouldn't have access.
- Require `plan` output review in a PR/CI gate before any `apply` runs against a `prod`-tagged backend key or workspace.

- A wrapper script or Makefile target that echoes the current workspace/backend and requires typing the environment name to confirm, the way `terraform destroy` itself makes you type "yes."

Immediate response: don't panic-apply. First, check whether state still reflects what was destroyed (`terraform state list` , `terraform show`) — if the destroy is mid-flight or partially failed, know exactly what's gone versus what's still there before doing anything else. Then restore from the most recent known-good state (if using S3 with versioning, the *previous* state file version tells you what existed) and rebuild via `apply` , or restore from infrastructure backups/snapshots for anything stateful (databases, persistent volumes) that Terraform itself can't "undo" — Terraform only tracks infrastructure shape, not data.

Q3. You inherited a Terraform codebase where `terraform.tfstate` is committed to Git. What's wrong with that, and how do you fix it without breaking anything currently running?

What's wrong: state holds real resource IDs and every attribute value in plaintext, including anything marked `sensitive` in config (that flag only masks CLI output, not what's written to the file). It's also a guaranteed source of merge conflicts and lost updates the moment two people touch it in parallel branches.

Fix, without an outage:

1. Set up a backend (S3 + DynamoDB, or equivalent) — don't touch the running resources yet.
2. From the directory with the currently-committed state, add the `backend` block and run `terraform init`. Terraform detects the backend change and offers to **migrate** existing local state into the new backend — say yes. This copies state; it does not touch any real infrastructure.
3. Run `terraform plan` immediately after migrating and confirm it shows **no changes**. If it does, stop and investigate before applying anything — that means the migration didn't cleanly preserve what Terraform thinks exists.
4. Delete the committed state file from the repo, add `*.tfstate*` to `.gitignore`.
5. Purge the file from Git history (`git filter-repo`), since anyone with a clone still has it. Rotate any credentials or secrets that were visible in that state, since a public or widely-cloned repo means assume it's already been seen.

Q4. Two engineers each ran `terraform apply` at almost the same time against a backend with no locking configured. What actually happens to the state, and how do you architect around it?

Without locking, both processes read the same state at roughly the same moment, both compute a plan against that snapshot, both apply, and both write their resulting state back — the second write **wins entirely**, silently discarding whatever the first apply changed from Terraform's point of view. The real infrastructure the first apply created still exists in AWS; Terraform's state file simply forgets about it. The next `plan` then proposes to *recreate* those "missing" resources, often colliding with the real ones (name conflicts) or worse, succeeding and creating duplicates.

The fix: a locking backend. Classic setup is S3 + a DynamoDB table (`dynamodb_table` in the backend block) acting as a distributed lock — Terraform acquires a row in that table before reading state and releases it after writing. Terraform ≥ 1.10 can instead lock using a plain object in the S3 bucket itself (`use_lockfile = true`), removing the need for the DynamoDB table entirely. Either way, the second `apply` gets a clean, immediate "state lock" error instead of a silent, undetected data-loss race.

Q5. `terraform plan` shows Terraform wants to destroy and recreate an RDS instance that nobody touched in the `.tf` files. What are the likely causes, and how do you diagnose it?

This is **drift** or a subtle dependency change, not usually a direct edit. Diagnosis order:

1. `terraform plan` output itself — it shows *which attribute* triggered the replacement (marked with `# forces replacement`). Start there rather than guessing.
2. Check if the attribute is something that can only be set at creation time (e.g., `identifier` , `availability_zone` for some resource types) — if someone changed it manually in the AWS console, or a module upgrade changed a default, Terraform sees a diff on an immutable attribute and the only way to reconcile is destroy + recreate.
3. Check if a `data` source feeding into this resource resolved to a new value — e.g., an AMI `data.aws_ami` lookup with `most_recent = true` picking up a newly-published AMI changes the `ami` argument every time a new one is released, even though nobody edited the `.tf` file.
4. Check for a provider version bump that changed a default value for an unset argument.
5. Run `terraform plan -refresh-only` first to see if this is actually *state drift* (someone changed it out-of-band in the console) versus a genuine config-driven diff.

Once identified: if it's an unwanted destroy-recreate on something stateful (a database!), consider `ignore_changes` on that specific attribute, or fix the root config to match reality, or if it's genuinely a manual out-of-band change, decide whether to import the manual change into state (accept it) or plan to revert it.

Modules

Q6. You're asked to write a module for an S3 bucket that will be used by 15 different teams with very different requirements. How do you design the interface so it doesn't turn into an unmaintainable pile of conditionals?

Start from what's actually optional versus what should always be true. Security defaults (encryption, public access block) shouldn't be toggleable at all in most orgs — bake them in, don't expose a variable that lets a team accidentally disable encryption. Then expose the genuine variation points as typed variables with sensible defaults:

`enable_versioning`, `lifecycle_expiration_days`, `force_destroy`, `tags`.

Resist adding a boolean for every possible feature — that's how you end up with 40 mutually-interacting flags. Instead:

- Group related settings into an `object` type variable with `optional()` fields and defaults, rather than 8 separate top-level variables.
- If two teams need genuinely different *shapes* of bucket (one needs replication, one needs a static-website config), that's a signal for two thin wrapper modules calling one shared core module, rather than one module trying to do everything via flags.
- Output everything a caller might plausibly need (`bucket_id`, `bucket_arn`, `bucket_regional_domain_name`) even if only some teams use each one — outputs are cheap to add up front, expensive to retrofit once 15 teams depend on the module.

Document with a table of inputs/outputs in the module's README so nobody has to read the `.tf` source to use it.

Q7. A shared module you maintain needs a breaking change — renaming a variable. 15 teams call this module. How do you roll it out without breaking everyone simultaneously?

Never rename in place on the version everyone's pinned to. Instead:

1. **Add the new variable alongside the old one**, with the module internally preferring the new one if set, falling back to the old one with a `deprecated` note in its description if not. Ship this as a *minor* version bump (if pre-1.0, or a backward-compatible major if using semver strictly) — nobody's calls break.
2. Communicate the deprecation (README, changelog, direct message to the calling teams) with a concrete removal version and date.
3. Cut a new major version (`v2.0.0`) that actually removes the old variable, after teams have had time to migrate. Because remote module sources are typically pinned with `?ref=v1.x.x`, existing callers are **completely unaffected** until they deliberately bump their pinned ref — that's the entire point of pinning.
4. Provide a migration snippet in the changelog showing the before/after variable name.

The core principle: **pinned module versions mean you can never truly "break" an existing caller** — you can only make v2 available and let each team upgrade on their own schedule.

Q8. You see a module with a hardcoded `provider "aws" { region = "us-east-1" }` block inside it. Why is this a problem, and what should the module do instead?

A module with its own `provider` block can only ever be used with that literal configuration — it can't be called twice for two regions in the same root module, can't be handed an aliased provider from the caller, and breaks entirely for anyone whose AWS setup uses a different auth mechanism (SSO profile, assumed role, GovCloud partition) than whatever was hardcoded.

The fix: **modules should never declare provider configuration**. They declare `required_providers` in a `versions.tf` (so Terraform knows which provider and version constraint the module needs), and every resource inside simply uses whichever provider the *calling* configuration passes down — implicitly (same provider name, default alias) or explicitly via the module's `providers` argument for multi-region/multi-account cases:

```
module "east_bucket" {  
  source = "../modules/s3"  
  providers = { aws = aws.us_east }  
}
```

Q9. Your team's Terraform apply is taking 20+ minutes because one giant root module manages 200+ resources. How do you approach breaking it apart?

Large monoliths are slow (every `plan` refreshes every resource) and risky (one bad change has a blast radius of the entire infrastructure). The fix is splitting into multiple independently-applied **stacks**, not just more modules within the same state:

1. Identify natural boundaries with different change frequency and different owners — e.g., "network" (VPC, subnets — changes rarely) versus "application" (deploys often) versus "shared data" (databases — changes rarely, different team owns it).
2. Give each boundary its own root configuration and its own state file/backend key.
3. Wire them together with `terraform_remote_state` data sources reading each other's outputs, rather than one shared state.
4. This means a routine app deployment's `plan` only refreshes the app stack's resources — seconds, not tens of minutes — and a mistake in the app stack can't accidentally touch the VPC.

The tradeoff to be upfront about: you lose the ability to see a single unified `plan` across everything at once, and you introduce ordering dependencies between applies (network stack must exist before the app stack that reads its outputs). Many teams layer a tool like Terragrunt on top specifically to manage that multi-stack orchestration.

Workspaces

Q10. Explain, as if to someone who's only used one Terraform environment before, why `terraform workspace new prod` isn't the same as having a proper production environment.

A workspace only isolates the **state file** — the record of what was built. It does nothing to isolate the AWS account, the IAM permissions used to apply, the backend location, or which humans/CI jobs are allowed to touch it. If dev and prod are the same AWS account behind different workspaces, a `plan -target` typo or a bad module upgrade can just as easily touch prod resources as dev ones, because the credentials running Terraform have access to both — the workspace boundary is enforced by *you remembering to select correctly*, not by the cloud provider.

A "proper" production environment usually means a **separate AWS account entirely**, with its own IAM boundary, so that even a completely wrong `apply` physically cannot reach production because the credentials in use don't have access to that account at all. Workspaces are the right tool when environments genuinely are just size variants within one account and one team's blast-radius tolerance; they're the wrong tool when "prod" needs a hard security boundary, which is most real organizations' actual requirement.

Q11. You add a fourth environment, `qa`, using `terraform workspace new qa`, run `apply`, and it fails because an S3 bucket name is already taken. What went wrong in the original design, and how do you fix it going forward?

The original config almost certainly hardcoded a bucket name, or built one from something that doesn't vary per workspace, e.g. `bucket = "myapp-bucket"` instead of incorporating `terraform.workspace`. Since S3 bucket names are globally unique, `dev` and `stg` might've been fine as long as they had distinct literal names, but a new `qa` workspace either collides with an existing name or exposes that the original names weren't actually parameterized by environment at all.

Fix: any globally-unique name must incorporate `terraform.workspace` (or an explicit `var.env`, whichever the codebase uses as its source of truth) — `bucket = "myapp-${terraform.workspace}-bucket"`. More broadly, this is a sign to audit the whole config for anything that assumed a fixed small set of environments rather than deriving names/settings from the current workspace or an explicit per-environment map.

Q12. Your `locals` block maps `terraform.workspace` to a sizing config, but someone typos a workspace name

(`prod` vs `prodd`) when running `terraform workspace new`. What happens, and how do you guard against it?

`terraform workspace new prodd` succeeds — Terraform has no idea what workspace names are "valid" for your config, it just creates a new empty state under that name. If your `locals.cfg = lookup(local.env_config, terraform.workspace, some_default)` falls back to a default on an unrecognized key, the `apply` will proceed silently using default (often dev-sized) settings under a workspace that looks like it should be production.

Guard rails:

```
resource "terraform_data" "workspace_guard" {
  input = terraform.workspace
  lifecycle {
    precondition {
      condition     = contains(keys(local.env_config), terraform.workspace)
      error_message = "Unknown workspace '${terraform.workspace}'. Not in local.env_config."
    }
  }
}
```

This fails the `plan` outright on an unrecognized workspace rather than silently falling back to a default. A `check` block can additionally *warn* (without blocking) on suspicious combinations, like a workspace containing "prod" not matching the expected sizing tier.

`count`, `for_each`, and `lifecycle`

Q13. A resource block uses `count = length(var.subnet_ids)`. Someone removes the second element from that list. What does Terraform propose to do, and why is that a problem?

Because `count`-based resources are addressed by numeric index (`aws_instance.x[0]`, `[1]`, `[2]`), removing the middle element of the underlying list shifts every subsequent element's index down by one. Terraform's plan will show: index `[1]` (previously the second subnet) is now associated with what *used to be* index `[2]`'s subnet — from Terraform's perspective, that's a change to an existing resource's argument (or in many cases, a forced replacement), and every index after the removed one gets destroyed and recreated even though conceptually only one thing was actually removed.

The fix: convert to `for_each` keyed by something stable (the subnet ID itself, or a name), so removing one entry only affects the resource instance tied to that specific key — everything else is left completely alone.

Q14. You need to replace an EC2 instance's AMI with zero downtime — it sits behind a load balancer. What Terraform mechanism do you reach for, and what else has to be true for it to actually work?

`lifecycle { create_before_destroy = true }`. Without it, Terraform's default order is destroy-then-create: the old instance is terminated, *then* the new one is built — guaranteed downtime while nothing is serving traffic.

With `create_before_destroy`, Terraform builds the new instance first, and only destroys the old one once the new one exists in state. But that alone doesn't guarantee zero downtime for real traffic — for that, the new instance also needs to be:

- Attached to the load balancer / target group *before* the old one is detached (usually handled automatically if the target group attachment is a separate resource referencing the instance, since Terraform will create the new attachment before destroying the old instance-attachment pair, given the dependency graph).
- Passing health checks before the LB starts sending it real traffic — this is enforced by the LB/target group's own health check configuration, not by Terraform itself; Terraform only controls the *infrastructure* lifecycle, not application readiness.

Also worth mentioning: any resource whose name must be globally/locally unique (like a fixed `Name` tag used as a uniqueness key elsewhere) can't use `create_before_destroy` naively, since the new and old resource would collide on that name while both exist momentarily — sometimes this requires a name suffix or `count` / `for_each` key change too.

Q15. What's the difference between a `precondition`, a `postcondition`, and a `check` block, and when would you use each?

	Runs	Failure blocks	Use for
<code>precondition</code> (inside <code>lifecycle</code>)	Before the resource is created/updated, during plan	The plan/apply	Validating <i>inputs</i> to this specific resource are sane before Terraform even tries — e.g., an AMI must exist in the current region
<code>postcondition</code> (inside <code>lifecycle</code>)	After the resource is created/updated	The apply	Validating the <i>result</i> actually matches expectations — e.g., an instance actually got a public IP when one was requested
<code>check</code> (top-level block)	After apply, can include its own <code>data</code> sources	Nothing — only emits a warning	Ongoing health/sanity assertions that shouldn't block deployment — e.g., "the website returns 200" — useful as a lightweight, code-reviewed smoke test that runs on every apply

Interview framing that lands well: preconditions and postconditions are about *this resource's correctness*; `check` is closer to a monitoring assertion embedded in your IaC, intentionally non-blocking because you don't want a flaky external health check to prevent an otherwise-correct infrastructure change from landing.

Import, drift, and real-world messiness

Q16. You've been handed an AWS account with hundreds of manually-created resources and told to "put it under Terraform." How do you approach this without an outage?

This is a common real scenario and interviewers want to see a methodical, low-risk plan rather than "just write the config and import everything at once."

1. **Inventory first.** Use `aws` CLI / Config / a tool like `terraformer` to enumerate what actually exists, rather than trusting documentation.
2. **Prioritize by risk and blast radius**, not alphabetically — start with something low-stakes (an S3 bucket used for logs) to validate the workflow before touching anything load-bearing.
3. **Write the `.tf` resource block first**, matching the real resource's current configuration as closely as possible.
4. **Use `terraform import`** (or an `import` block + `terraform plan -generate-config-out=`) to bring it into state.
5. **Run `terraform plan` immediately and read every line.** The goal is **zero proposed changes** — if the plan wants to change something, your `.tf` doesn't yet match reality, and applying now would make a live change to a resource nobody intended to touch. Iterate on the `.tf` until the plan is clean.
6. Only once the plan is clean does this resource "belong" to Terraform. Repeat, resource by resource or in small logical batches, never all 300 resources in one giant sweep where a single mistake is hard to isolate.
7. Once everything's imported, `terraform state list` becomes your source of truth for what's covered — anything not listed is still manually managed and still a risk.

Q17. `terraform plan` shows no changes, but you know someone manually changed a security group rule in the console yesterday. Why might Terraform not be seeing it?

A few real possibilities, in likely order:

1. **State hasn't been refreshed.** Older Terraform versions and some workflows skip an implicit refresh; run `terraform apply -refresh-only` (or, on very old versions, `terraform refresh`) explicitly and see if it now detects a difference.
2. **The changed attribute isn't managed by this Terraform resource at all** — e.g., if security group rules were created as separate `aws_security_group_rule` / `aws_vpc_security_group_ingress_rule` resources but the manual change was to a rule that was never actually defined in the `.tf` (an "orphan" rule added directly in the console), Terraform has no resource address for it and will never surface it in a plan, because it doesn't know that rule is supposed to be under management.
3. **`ignore_changes` includes the relevant attribute** — check the resource's `lifecycle` block; if the attribute is deliberately ignored, drift there is invisible to `plan` by design.
4. The change may have been reverted or overwritten by something else since — worth confirming the console still shows the manual change.

Q18. Your CI pipeline runs `terraform apply -auto-approve` on every merge to `main`. What could go wrong with this, and what guardrails would you add?

The core risk: `-auto-approve` means nobody reviews the actual plan before it executes — a plan is only as safe as its diff, and skipping human review of that diff on every single merge means a subtle bug (a bad variable default, an unintended resource deletion from a refactor) applies to real infrastructure with zero human check.

Guardrails, roughly in order of value:

1. **Plan on PR, apply on merge** — run `terraform plan` as a PR check and post the output as a comment, so reviewers see the actual infrastructure diff alongside the code diff before approving.
2. **Require the applied plan to be the exact plan that was reviewed** — save the plan artifact from the PR stage (`terraform plan -out=tfplan`) and apply that exact file after merge, rather than re-planning and blindly trusting it matches; this closes the gap where something changes between review and merge.
3. **Restrict what CI's credentials can do** — least-privilege IAM so a bad apply has a bounded blast radius (e.g., app-stack CI credentials can't touch network or IAM resources).
4. **Manual approval gate for destructive changes** — many CI systems can pattern-match plan output for `# forces replacement` or resource deletions and require a human click before those specific applies proceed, while safe additive changes flow through automatically.
5. **prevent_destroy on the truly irreplaceable resources** as a last-resort backstop, independent of pipeline logic.

Terraform vs. broader context

Q19. When would you deliberately choose *not* to use Terraform for something, even in an otherwise fully Terraform-managed environment?

A few honest cases interviewers like hearing:

- **Highly dynamic, application-owned scaling decisions** — e.g., exact desired-count adjustments for an autoscaling group driven by real-time load are better left to the ASG's own scaling policies (or an application-level orchestrator) than fought over between Terraform and a live autoscaler; use `ignore_changes` on `desired_capacity` and let the ASG own that number.
- **Secrets that need frequent rotation** — Terraform can *create* a Secrets Manager entry, but ongoing rotation is usually better handled by the secret store's native rotation Lambda than by re-running `apply` every time a password needs to change.
- **One-off, exploratory, or throwaway infrastructure** — spinning up a quick sandbox VM to test something for twenty minutes doesn't need a `.tf` file, state tracking, and a PR.
- **Application deployment itself** (pushing new container images/code) — Terraform manages the *shape* of infrastructure well; a dedicated deploy tool (Argo CD, a CI/CD pipeline, `kubectl rollout`) is usually the better fit for the actual "ship the new version" step, with Terraform stopping at "here's the cluster/service that exists" and GitOps taking over from there.

Q20. Explain the difference between `terraform plan`'s "no changes" and infrastructure actually matching the code, in a case where they might diverge.

`terraform plan` compares the **desired state (your `.tf` files)** against the **last-known state (the state file)**, after an implicit refresh reconciles state against the real provider APIs for attributes Terraform tracks. "No changes" means those two things agree — it does **not** guarantee the real resource has no other properties or hasn't been touched in ways Terraform's provider schema doesn't model.

Concrete divergence case: someone manually adds an inline policy to an IAM role that Terraform created, but Terraform's `aws_iam_role` resource in this configuration doesn't manage inline policies at all (they were added via a separate `aws_iam_role_policy` resource that was never written). `plan` shows no changes, because from Terraform's point of view nothing it's tracking has changed — but the real-world role now has permissions nobody defined in code, and Terraform will never surface or reconcile that until someone writes a matching resource for it (or the "no changes" answer is technically true but practically incomplete).

Interview framing: **Terraform's model of correctness is "does reality match what I was told to track," not "does reality match everything about this resource."** Anything not represented as a resource attribute in the schema is invisible to it — a genuinely important limitation to understand rather than assume away.

12. Rapid-Fire Definitions (for quick review before an interview)

Term	One-line definition
State	JSON record mapping config resource addresses to real-world object IDs and attributes
Backend	Where state is stored and how it's locked (local, S3, Terraform Cloud, etc.)
Provider	Plugin that translates <code>.tf</code> resources into API calls for a specific platform (AWS, Azure, Kubernetes...)
Module	A reusable directory of <code>.tf</code> files, callable with a <code>module</code> block
Root module	The top-level configuration you run <code>apply</code> from
Workspace	A named, isolated state file for the same configuration
Drift	Real infrastructure no longer matches what Terraform's state says it built
Plan	A dry-run diff between desired config and current state
Taint (legacy) / <code>-replace</code>	Force a specific resource to be destroyed and recreated on the next apply
Data source	Read-only lookup of information, doesn't create/manage anything
Provisioner	Escape hatch to run a script on a resource at create/destroy time (last resort — prefer native provider features or config management tools)
HCL	HashiCorp Configuration Language — the <code>.tf</code> file syntax
Idempotency	Applying the same config repeatedly produces the same end state, without duplicating resources
Blast radius	How much infrastructure a single mistaken change could affect
State locking	Preventing two concurrent applies from corrupting the same state file

End of notes.